Comandos de Linux muy usados

Los 40 comandos de Linux más utilizados

Veamos los comandos básicos de Linux más usados y sus ejemplos para la administración del sistema.

1. Comando ls

El comando **ls** lista los archivos y directorios de tu sistema. Ésta es la sintaxis:

`ls [/directorio/carpeta/ruta]`

```
root@srv:/# ls /directory/folder/path
file1.txt
```

Si eliminas la ruta, el comando **ls** mostrará el contenido del directorio de trabajo actual. Puedes modificar el comando utilizando estas opciones:

- **-R:** Lista todos los archivos de los subdirectorios.
- **-a:** Muestra todos los archivos, incluidos los ocultos.
- **-lh:** convierte los tamaños a formatos legibles, como **MB**, **GB** y **TB**.

2. Comando cd

Utiliza el comando **cd** para navegar por los archivos y directorios de Linux. Para utilizarlo, ejecuta esta sintaxis con privilegios sudo:

`cd /directorio/carpeta/ruta`

```
root@srv:/# cd /directory/folder/path
root@srv:/directory/folder/path#
```

Dependiendo de tu ubicación actual, requiere la ruta completa o el nombre del directorio. Por ejemplo, omite **/nombredeusuario** de **/nombredeusuario/directorio/carpeta** si ya estás dentro de él.

Si omites los argumentos, irás a la carpeta de inicio. Aquí tienes algunos atajos de navegación:

- **cd ~[nombredeusuario]:** va al directorio personal de otro usuario.
- **cd ..**: mueve un directorio hacia arriba.
- **cd-**: cambia al directorio anterior.

3. Comando pwd

El comando **pwd** imprime la ruta de tu directorio de trabajo actual, como **/inicio/directorio/ruta**. Ésta es la sintaxis del comando:

```
pwd [opción]
```

Admite dos opciones. La opción **-L** o **–logical** imprime el contenido de las variables de entorno, incluidos los enlaces simbólicos. Por su parte, **-P** o **-physical** muestra la ruta real del directorio actual.

```
root@srv:/directory/folder/path# pwd
/directory/folder/path
```

4. Comando mkdir

Utiliza el comando **mkdir** para crear uno o varios directorios y establecer sus permisos. Asegúrate de que estás autorizado a crear una nueva carpeta en el directorio padre. Ésta es la sintaxis básica:

```
mkdir [opción] [nombre_directorio]
```

Para crear una carpeta dentro de un directorio, utiliza la ruta como parámetro del comando. Por ejemplo, **mkdir música/canciones** creará una carpeta **canciones** dentro de **música**. Aquí tienes varias opciones comunes del comando mkdir:

- **-p:** crea un directorio entre dos carpetas existentes. Por ejemplo, **mkdir -p Música/2025/Canciones** crea un nuevo directorio **202**5.
- **-m:** establece los permisos de la carpeta. Por ejemplo, introduce **mkdir -m777 directory** para crear un directorio con permisos de lectura, escritura y ejecución para todos los usuarios.
- **-v:** imprime un mensaje por cada directorio creado.

```
root@srv:/# mkdir -v new-folder
mkdir: created directory 'new-folder'
```

5. Comando rmdir

Utiliza el comando **rmdir** para borrar un directorio vacío en Linux. El usuario debe tener privilegios **sudo** en el directorio padre. Esta es la sintaxis:

```
rmdir [opción] nombre_directorio
```

Si la carpeta contiene un subdirectorio, el comando devolverá un error. Para forzar la eliminación de un directorio no vacío, utiliza la opción **-p**.

6. Comando rm

Utiliza el comando rm para eliminar permanentemente los archivos de un directorio. Ésta es la sintaxis general:

```
rm [nombrearchivo1] [nombrearchivo2] [nombrearchivo3]
```

Ajusta el número de archivos del comando según tus necesidades. Si encuentras un error, asegúrate de que tienes permiso de **escritura** en el directorio.

Para modificar el comando, añade las siguientes opciones:

- **-i:** pide una confirmación antes de borrar.
- **-f:** permite eliminar archivos sin confirmación.
- **-r:** borra archivos y directorios recursivamente.

¡Atención! Utiliza el comando **rm** con precaución, ya que el borrado es irreversible. Evita utilizar las opciones **-r** y **-f** ya que pueden borrar todos tus archivos. Añade siempre la opción **-i** para evitar el borrado accidental.

7. Comando cp

Utiliza el comando **cp** para copiar archivos o directorios, incluido su contenido, de tu ubicación actual a otra. Tiene varios casos de uso, como:

- Copiar un archivo del directorio actual a otra carpeta. Especifica el nombre del archivo y la ruta de destino:

`cp nombrearchivo.txt /inicio/nombredeusuario/documentos`

- Duplicar varios archivos en un directorio. Introduce los nombres de los archivos y la ruta de destino:

`cp nombrearchivo1.txt nombrearchivo2.txt nombrearchivo3.txt /inicio/nombredeusuario/documentos`

- Copiar el contenido de un fichero a otro dentro del mismo directorio. Introduce el fichero de origen y el de destino:

`cp nombrearchivo1.txt nombrearchivo2.txt`

- Duplicar un directorio entero. Pasa la bandera **-R** seguida del directorio de origen y de destino:

`cp -R /inicio/nombredeusuario/documentos /inicio/nombredeusuario/documentos_respaldo`

8. Comando mv

Utiliza el comando **mv** para mover o renombrar archivos y directorios. Para mover elementos, introduce el nombre del archivo seguido del directorio de destino:

mv nombrearchivo.txt /inicio/nombredeusuario/documentos

Mientras tanto, utiliza la siguiente sintaxis para renombrar un archivo en Linux con el comando **mv**:

`mv nombre_archivo_antiguo.txt nombre_archivo_nuevo.txt`

9. Comando touch

El comando **touch** te permite crear un archivo vacío en una ruta de directorio específica. Esta es la sintaxis:

```
touch [opción] /inicio/directorio/ruta/archivo.txt
```

Si omites la ruta, el comando creará el elemento en la carpeta actual. También puedes utilizar **touch** para generar y modificar una marca de tiempo en la línea de comandos de Linux.

10. Comando file

El comando file te permite comprobar un tipo de archivo, ya sea texto, imagen o binario. Ésta es la sintaxis:

```
file nombrearchivo.txt
```

Para comprobar en bloque varios archivos, enuméralos individualmente o utiliza su ruta si están en el mismo directorio. Añade la opción **-k** para mostrar información más detallada e **-i** para mostrar el tipo MIME del archivo.

11. Comando find

Utiliza el comando **find** para buscar archivos dentro de un directorio concreto. Ésta es la sintaxis:

```
find [opción] [ruta] [expresión]
```

Por ejemplo, para buscar un archivo llamado **archivo1.txt** dentro de la carpeta **directorio** y sus subdirectorios, utiliza este comando:

```
find /inicio -name archivo1.txt
```

```
root@srv:/# find /directory -name file1.txt
/directory/folder/path/file1.txt
```

Si omites la ruta, el comando buscará en el directorio de trabajo actual. También puedes buscar directorios utilizando lo siguiente:

```
find ./ -type d -name nombredirectorio
```

12. Comando locate

El comando **locate** te permite encontrar un archivo en el sistema de base de datos. Añade la opción **-i** para desactivar la distinción entre mayúsculas y minúsculas, y un asterisco (*) para buscar contenido con varias palabras clave. Por ejemplo:

```
locate -i escuela*nota
```

El comando busca archivos que contengan **escuela** y **nota**, independientemente de la mayúscula o minúscula que tengan.

13. Comando tar

El comando tar archiva varios elementos en un archivo **TAR**, un formato similar al **ZIP** con compresión opcional. Ésta es la sintaxis

`tar [opciones] [fichero_archivo] [archivo de destino o directorio]`

Por ejemplo, introduce lo siguiente para crear un nuevo archivo **nuevoarchivo.tar** en el directorio **/inicio/usuario/documentos**:

`tar -cvzf nuevoarchivo.tar /inicio/usuario/documentos`

14. Comandos zip y unzip

El comando **zip** te permite comprimir elementos en un archivo **ZIP** con la relación de compresión óptima. Ésta es la sintaxis:

`zip [opciones] archivozip archivo1 archivo2....`

Por ejemplo, este comando comprime **nota.txt** en **archivo.zip** en el directorio de trabajo actual:

`zip archivo.zip nota.txt`

Utiliza el comando **unzip** para extraer el archivo comprimido. Ésta es la sintaxis:

`unzip [opción] nombre_archivo.zip`

15. Comando cat

Concatenar o **cat** es uno de los comandos Linux más utilizados. Lista, combina y escribe el contenido de los archivos en la salida estándar. Ésta es la sintaxis:

`cat nombrearchivo.txt`

Hay varias formas de utilizar el comando **cat**:

- **`cat > filen.txt`:** crea un nuevo archivo.
- **`cat archivo1.txt archivo2.txt > archivo3.txt`:** fusiona el **archivo1.txt** con **el archivo2.txt** y almacena el resultado en **el archivo3.txt**.
- `tac archivo.txt` : muestra el contenido en orden inverso.

16. Comando head

El comando head imprime las diez primeras líneas de un archivo de texto o datos canalizados en tu interfaz de línea de comandos. Ésta es la sintaxis general:

`head [opción] [archivo]`

Por ejemplo, para ver las diez primeras líneas de **nota.txt** en el directorio actual, introduce:

```
head nota.txt
```

El comando **head** acepta varias opciones, como:

- **-n:** cambia el número de líneas impresas. Por ejemplo, **head -n 5** muestra las cinco primeras líneas.
- **-c:** imprime el primer número personalizado de bytes del archivo.
- **-q:** desactiva las cabeceras que especifican el nombre del archivo.

```
root@srv:/directory/folder/path# head -n 5 file1.txt
This is the first line
This is the second line
This is the third line
This is the fourth line
This is the fifith line
```

17. Comando tail

El comando **tail** muestra las diez últimas líneas de un archivo, lo que resulta útil para comprobar nuevos datos y errores. Ésta es la sintaxis:

```
tail [opción] [archivo]
```

Por ejemplo, introduce lo siguiente para mostrar las diez últimas líneas del archivo **colores.txt**:

```
tail -n colores.txt
```

18. Comando grep

La **expresión regular global** o comando **grep** te permite encontrar una palabra buscando en el contenido de un archivo. Este comando de Linux imprime todas las líneas que contienen las cadenas coincidentes, lo que resulta útil para filtrar archivos de registro de gran tamaño.

Por ejemplo, para mostrar las líneas que contienen **azul** en el archivo **blocdenotas.txt**, introduce:

```
grep blue blocdenotas.txt
```

19. Comando awk

El comando **awk** escanea patrones de expresiones regulares en un archivo para recuperar o manipular datos coincidentes. Ésta es la sintaxis básica:

```
awk '/regex pattern/{action}' archivo_entrada.txt
```

La acción puede ser operaciones matemáticas, sentencias condicionales como **if**, expresiones de salida como **print** y un comando de **borrado**. También contiene la notación **$n**, que se refiere a un

campo de la línea actual.

Para añadir varias acciones, enuméralas según el orden de ejecución, separadas mediante punto y coma. Por ejemplo, este comando contiene sentencias matemáticas, condicionales y de salida:

```
awk -F':' '{ total += $2; estudiantes[$1] = $2 } END { average = total /
length(estudiantes); print "Average:", average; print "Above average:"; for
(estudiante en estudiantes) if (estudiantes[estudiante] > average) print
estudiante }' puntuación.txt`
```

```
root@srv      :/directory/folder/path# awk -F':' '{ total += $2; students[$1] =
$2 } END { average = total / length(students); print "Average:", average; print
"Above average:"; for (student in students) if (students[student] > average) pri
nt student }' score.txt
Average: 80.5
Above average:
Student B
Student C
```

20. Comando sed

El comando **sed** te permite encontrar, sustituir y eliminar patrones en un archivo sin utilizar un editor de texto. Ésta es la sintaxis general:

```
sed [opción] 'script' archivo_entrada
```

El script contiene el patrón de expresión regular buscado, la cadena de sustitución y los subcomandos. Utiliza el subcomando **s** para reemplazar los patrones coincidentes o **d** para eliminarlos.

Al final, especifica el archivo que contiene el patrón a modificar. Aquí tienes un ejemplo de comando que sustituye el **rojo** de **colores.txt** y **tono.txt** por el **azul**:

```
sed 's/red/blue' colores.txt tono.txt
```

21. Comando chmod

El comando **chmod** modifica los permisos de directorio o archivos en linux. Ésta es la sintaxis básica:

```
chmod [opción] [permiso] [nombre_archivo]
```

En Linux, cada archivo está asociado a tres clases de usuario: **propietario**, **miembro de grupo** y **otros**. También tiene tres permisos: **lectura**, **escritura** y **ejecución**.

Si un propietario quiere conceder todos los permisos a todos los usuarios, el comando tiene el siguiente aspecto:

```
chmod -rwxrwxrwx nota.txt
```

22. [Comando chown]

El comando **chown** te permite cambiar la propiedad de un archivo, directorio o enlace simbólico al nombre de usuario especificado. Esta es la sintaxis:

```
chown [opción] owner[:grupo] archivo(s)
```

Por ejemplo, para que **usuariolinux2** sea el propietario de **nombrearchivo.txt**, utiliza:

```
chown usuariolinux2 nombrearchivo.txt
```

23. Comando sudo

El **superusuario do** o **sudo** es uno de los comandos más básicos de Linux. Ejecuta tu comando con permisos administrativos o de root. Esta es la sintaxis general:

```
sudo (comando)
```

Cuando ejecutes un comando sudo, Terminal te pedirá la contraseña de root. Por ejemplo, este fragmento ejecuta **useradd** con el privilegio de superusuario:

```
sudo useradd nombredeusuario
```

También puedes añadir una opción, como por ejemplo

- **-k:** invalida el archivo de marca de tiempo.
- **-g:** ejecuta comandos como un nombre o ID de grupo especificado.
- **-h:** ejecuta comandos en el host.

¡Atención! Ejecutar un comando con privilegios sudo puede modificar todos los aspectos de tu sistema. Puesto que su uso indebido puede romper tu sistema, ejecuta el comando con precaución y sólo si comprendes sus posibles consecuencias.

24. Comando ps

El comando ps crea una instantánea de todos los procesos en ejecución de tu sistema. Ejecutándolo sin una opción o argumento se listarán los procesos en ejecución en el intérprete de comandos con la siguiente información:

- ID único del proceso (**PID**).
- Tipo de terminal (**TTY**).
- Duración (**TIME**).
- Comando que lanza el proceso (**CMD**).

El comando **ps** acepta varias opciones, entre ellas:

- **-T:** muestra todos los procesos asociados a la sesión shell actual.
- **-u nombredeusuario:** lista los procesos asociados a un usuario concreto.

- **-A:** Muestra todos los procesos en ejecución.

25. Comando top

El comando **top** muestra los procesos en ejecución y el estado del sistema en tiempo real, incluida la utilización de recursos. Ayuda a identificar los procesos que consumen muchos recursos, permitiéndote desactivarlos fácilmente.

26. Comando kill

Utiliza el comando **kill** para terminar un programa que no responde utilizando su número de identificación (PID). Para comprobar el PID, ejecuta el siguiente comando:

```
ps ux
```

Para detener el programa, introduce la siguiente sintaxis:

```
kill \[opcion_señal\] pid
```

Existen 64 señales para terminar un programa, pero **SIGTERM** y **SIGKILL** son las más comunes. **SIGTERM** es la señal por defecto que permite al programa guardar su progreso antes de detenerse. Mientras tanto, **SIGKILL** obliga a los programas a detenerse y descartar el progreso no guardado.

27. Comando df

Utiliza el comando **df** para comprobar el uso de espacio en disco de un sistema Linux en porcentaje y en kilobytes (**KB)**. Ésta es la sintaxis:

```
df [opciones] [archivo]
```

Si no especificas el elemento, este comando mostrará información sobre cada sistema de archivos montado. Estas son algunas opciones aceptables:

- **-m:** muestra información sobre el uso del sistema de archivos en **MBs**.
- **-k:** imprime el uso del sistema de archivos en **KBs**.
- **-T:** muestra el **tipo de** sistema de archivos en una nueva columna.

28. Comando du

Utiliza **du** para comprobar el consumo de almacenamiento de un archivo o directorio. Recuerda especificar la ruta del directorio cuando utilices este comando, por ejemplo:

```
du /inicio/usuario/documentos
```

El comando **du** tiene varias opciones, como:

- **-s:** muestra el tamaño total de la carpeta especificada.
- **-m:** proporciona información de carpetas y archivos en **MB.**

- **-k:** muestra la información en **KB**.
- **-h:** informa de la fecha de última modificación de las carpetas y archivos mostrados.

29. Comando uname

El comando **uname** o **nombre unix** imprime información sobre tu máquina, incluyendo su hardware, el nombre del sistema y el núcleo Linux. Ésta es la sintaxis básica:

```
uname [opción]
```

Aunque puedes utilizarlo sin opción, añade lo siguiente para modificar el comando:

- **-a:** imprime toda la información del sistema.
- **-s:** muestra el nombre del núcleo.
- **-n:** muestra el nombre de host del nodo del sistema.

```
root@srv:/# uname -asn
Linux srv       5.15.0-1038-kvm #43-Ubuntu SMP Mon Jul 17 19:45:17 UTC 2023 x86_
64 x86 64 x86 64 GNU/Linux
```

30. Comando htop

El comando **htop** es un programa interactivo para supervisar los recursos del sistema y los procesos del servidor. A diferencia de **top**, ofrece funciones adicionales como el manejo con el ratón e indicadores visuales. Ésta es la sintaxis del comando:

```
htop [opciones]
```

Admite opciones como:

- **-d:** muestra el retardo entre actualizaciones en décimas de segundo.
- **-C:** activa el modo monocromo.
- **-h:** muestra el mensaje de ayuda y salidas.

31. Comando ping

El comando **ping** es uno de los más utilizados en Linux. Te permite comprobar si se puede acceder a una red o a un servidor, lo que resulta útil para solucionar problemas de conectividad. Aquí tienes la sintaxis:

`ping [opción] [hostname_o_dirección_IP]`

Por ejemplo, ejecuta lo siguiente para comprobar la conexión y el tiempo de respuesta a **Google**:

`ping google.com`

32. Comando curl

El comando **curl** transfiere datos entre servidores. Su uso habitual es para recuperar el contenido de una página web en tu sistema utilizando su URL. Esta es la sintaxis:

`curl [opción] URL`

Sin embargo, puedes añadir varias opciones para modificar el comportamiento del comando **curl** para otras tareas. Algunas de las más populares son:

- **-o** o **-O:** descarga archivos desde una URL.
- **-X:** cambia el método HTTP GET por defecto.
- **-H:** envía una cabecera personalizada a la URL.
- **-F:** sube un archivo al destino especificado.

33. Comando wget

Utiliza el comando **wget** para descargar archivos de Internet mediante los protocolos **HTTP**, **HTTPS** o **FTP**. Esta es la sintaxis:

`wget [opción] [url]`

Por ejemplo, introduce lo siguiente para descargar wordpress.

`wget https://wordpress.org/latest.zip`

34. Comando scp

El comando **scp** copia de forma segura archivos o directorios entre sistemas a través de una red. Ésta es la sintaxis:

`scp [opción] [origen nombreusuario@IP]:/[directorio y nombre de archivo] [destino nombreusuario@IP]:/[directorio de destino].`

Para una máquina local, omite el nombre de host y la dirección IP. Utiliza las siguientes opciones para modificar el comportamiento de la copia:

- **-P:** cambia el puerto para copiar. El valor por defecto es **22**.
- **-l:** limita el ancho de banda del comando **scp**.
- **-C:** comprime los datos transferidos para hacerlos más pequeños.

35. Comando Watch

El comando **watch** permite al usuario ejecutar continuamente otra utilidad en un intervalo específico e imprimir los resultados como salida estándar. Ésta es la sintaxis:

`watch [opción] command`

Es útil para controlar los cambios en la salida de los comandos. Para modificar su **comportamiento**, utiliza las siguientes opciones:

- **-d:** muestra las diferencias entre ejecuciones de comandos.
- **-n:** cambia el intervalo predeterminado de dos segundos.
- **-t:** desactiva la cabecera que contiene el intervalo de tiempo, el comando, la marca de tiempo y el nombre de host.

36. Comando apt-get y apt

apt (y apt-get) es una herramienta de línea de comandos para manejar las bibliotecas de la Herramienta de Paquetes Avanzados (APT) en Linux basado en Debian, como Ubuntu. Requiere privilegios **sudo** o de **root**.

Este comando de Linux te permite gestionar, actualizar, eliminar e instalar software, incluidas sus dependencias. Ésta es la sintaxis principal:

`apt [opciones] (comando)`

Estos son los comandos más comunes que se utilizan con **apt**:

- **update:** sincroniza los archivos del paquete desde sus fuentes.
- **upgrade:** instala la última versión de todos los paquetes instalados.
- **check**: actualiza la caché de paquetes y comprueba las dependencias rotas.

```
root@srv:/# apt-get update
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Hit:3 https://dl.yarnpkg.com/debian stable InRelease
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
Hit:5 https://d17k9fuiwb52nc.cloudfront.net jammy InRelease
Get:6 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages
```

37. Comandos nano, vi y jed

Linux permite a los usuarios editar archivos utilizando un editor de texto como nano, **vi** o **jed**. Aunque la mayoría de las distribuciones incluyen **nano** y **vi**, los usuarios deben instalar **jed** manualmente. Todas estas herramientas tienen la misma sintaxis de comandos:

`nano nombrearchivo`

`vi nombrearchivo`

`jed nombrearchivo`

Si el archivo de destino no existe, estos editores crearán uno. Te recomendamos, por un lado, **nano** si quieres editar rápidamente archivos de texto. Por otro lado, utiliza **vi** o **jed** para scripts y programación.

38. Comando man

El comando **man** proporciona un manual de usuario de cualquier utilidad de la Terminal de Linux, incluyendo sus nombres, descripciones y opciones. Consta de nueve secciones:

- Programas ejecutables o comandos shell
- Llamadas al sistema
- Llamadas a la biblioteca
- Juegos
- Archivos especiales
- Formatos de archivo y convenciones
- Comandos de administración del sistema
- Rutinas del núcleo
- Varios

Ésta es la sintaxis del comando:

`man [opción] [número_sección] nombre_comando`

Si sólo utilizas el nombre del comando como parámetro, Terminal muestra el manual de usuario completo. Aquí tienes un comando de ejemplo para consultar la sección **1** del manual del comando **ls**:

`man 1 ls`

```
root@srv:/# man 1 ls
LS(1)                          User Commands                          LS(1)

NAME
       ls - list directory contents

SYNOPSIS
       ls [OPTION]... [FILE]...

DESCRIPTION
       List  information  about  the FILEs (the current directory by default).
       Sort entries alphabetically if none of -cftuvSUX nor --sort  is  speci
       fied.

       Mandatory  arguments  to  long  options are mandatory for short options
       too.
```

39. Comando history

Introduce **history** para listar los comandos ejecutados anteriormente. Te permite reutilizar los comandos sin reescribirlos. Para utilizarlo, introduce esta sintaxis con privilegios sudo:

`history [opción]`

```
root@srv:/# history
    1  sudo apt-get install node
    2  sudo apt-get install npm
    3  sudo apt-get install nodejs
    4  mkdir nodejsapp
```

Para volver a ejecutar una utilidad concreta, introduce un signo de exclamación (**!**) seguido del número de lista del comando. Por ejemplo, utiliza lo siguiente para volver a ejecutar el comando **255**:

`!255`

Este comando admite muchas opciones, como

- **-c:** borra la lista del historial.
- **-d offset:** borra la entrada del historial en la posición **OFFSET**.
- **-a:** añade líneas de historial.

40. Comando echo

El comando **echo** muestra una línea de texto como salida estándar. Ésta es la sintaxis básica del comando:

`echo [opción] [cadena]`

Por ejemplo:

`echo "Aprendo linux como un profesional en PPS"`

Este comando admite muchas opciones como:

- **-n:** muestra la salida sin la nueva línea final.
- **-e:** activa la interpretación de los siguientes escapes de barra invertida:
- **\b:** Elimina los espacios entre un texto.
- **\c:** no produce ningún otro resultado.